University of Trento

Scientific Programming

Midterm Simulation

Part A • 30 October 2026 • Duration: 90 minutes

General instructions

1. Download the exam zip file and extract it to your working directory.
2. Rename the extracted folder using the following naming convention:

   `sciprog-YYYY-MM-DD-firstname-lastname-ID`

   Replace each placeholder with your own details (the date is the exam date).

   Warning: not following the naming convention results in a deduction of 1 point from your grade.
3. Edit the files in that folder following the requirements of each exercise. Do not rename the provided functions or change their parameters.
4. At the end of the exam, compress the whole folder into a ZIP archive and submit it using the specified submission method. Include every file needed to run your code.

Important: Use only the Python standard library and the libraries you saw during the lectures, along with the built-in data structures (lists, tuples, dictionaries, sets) covered during the practical classes. Using other packages or libraries may affect your grade.

Checking your code with `unittest`

The exam folder contains `test_exercise{ID}.py`, a set of automatic tests written with Python's standard-library `unittest` module. Each test calls one of your functions on a small dataset whose correct answer is known and checks the result. Run them as often as you like while you work.

Running the tests

Open a terminal **inside the exam folder** (the one containing `exercise{ID}.py`) and run:

```
python -m unittest -v
```

Python finds every file whose name starts with `test`, runs each test and prints one line per test. The `-v` (verbose) flag shows the name of each test. To run only the tests for one task, give the test class name:

```
python -m unittest -v test_exercise{ID}.Test{NAME}
```

On some systems the command is `python3` instead of `python`.

Reading the result

Each test ends with one of three outcomes:

Outcome	Meaning
`ok`	The function returned the expected result.
`FAIL`	The function ran but returned a wrong result. The message shows the expected and the actual value.
`ERROR`	The function raised an exception, for example `NotImplementedError` because it is not written yet, or a `KeyError` from a misspelled dictionary key.

Example of a failing test

```
FAIL: test_fibonacci_number (test_exercise1.TestFibonacci)
----------------------------------------------------------
AssertionError: Lists differ: [0, 1, 1, 2, 3, 5, 8] != [0, 1, 1, 2, 3, 5, 123]

Ran 16 tests in 0.002s

FAILED (failures=1)
```

When every test passes, the last line reads `OK`.

Do not edit `test_exercise{ID}.py`: the original version is used for grading.

Before submitting, check that. . .

→ the script runs from the command line with `python exercise{ID}.py`;

→ all requested functions are implemented;

→ `python -m unittest -v` runs without errors;

→ the submission contains all required files.

1 Exercise 1 [Part A]

Write your code in `exercise1.py`. The file already contains the function signatures; replace each `raise NotImplementedError` with your implementation.

1.1 Dataset

The file `dataset/Carlsen.txt` contains chess games played by Magnus Carlsen between August 2024 and July 2026. Each game entry in the file is structured like this:

```
===NEW GAME===
[Event "Titled Tue 13th Aug Late"]
[Site "chess.com INT"]
[Date "2024.08.13"]
[Round "10"]
[White "Duda,J"]
[Black "Carlsen,M"]
[Result "0-1"]
[WhiteElo "2733"]
[BlackElo "2832"]
[ECO "C10"]

1.e4 e6 2.d4 d5 3.Nc3 h6 4.Nf3 Nf6 5.Bd3 c5 6.exd5 exd5 7.O-O c4 8.Re1+ Be7
...
51.Rg7+ Kf5 52.Rg8 Ke6  0-1
```

Notes on the format:

- `Result` is `1-0` (White wins), `0-1` (Black wins) or `1/2-1/2` (draw). The same result is repeated at the end of the moves.
- `ECO` is a code that identifies the opening played.
- The moves may span several lines. Each move number is followed by White's move and then Black's move. If the game ends on a White move, the last move number has no Black move.
- A ply is one move by one player, either White or Black. A full move consists of White's move followed by Black's move. For example, in `1.e4 e6`, `e4` is White's first ply and `e6` is Black's first ply. Together, they form the first full move.

1.2 Tasks

1) Define a function named `parse_games` that reads the file at `path` and **returns** a *list* with one *dictionary* per game, in file order. Each dictionary must have the following structure: **[10 points]**

```
{
    "event": "Titled Tue 13th Aug Late",
    "site": "chess.com INT",
    "date": "2024.08.13",
    "round": "10",
    "white": {"name": "Duda,J", "elo": 2733},
    "black": {"name": "Carlsen,M", "elo": 2832},
    "result": "0-1",
    "eco": "C10",
    "moves": [("e4", "e6"), ("d4", "d5"), ..., ("Rg8", "Ke6")],
}
```

The Elo ratings are integers; all other header values are strings. `"moves"` is a list of `(white_move, black_move)` tuples, one per move number, without move numbers or the result. If the game ends on a White move, the last tuple is `(white_move, None)`.

```
def parse_games(path):
    ...
    return games
```

2) Define a function named `results_by_color` that **returns** how many games `player` won, drew and lost with each colour, as a nested dictionary. **[3 points]**

```
def results_by_color(games, player="Carlsen,M"):
    ...
    return res
```

Expected result on the full dataset:

```
{'white': {'wins': 593, 'draws': 129, 'losses': 106},
 'black': {'wins': 530, 'draws': 156, 'losses': 121}}
```

3) Define a function named `top_opponents` that **returns** the `n` opponents `player` faced most often, as a list of `(name, games)` tuples sorted by number of games (highest first). Break ties by name in alphabetical order. **[3 points]**

```
def top_opponents(games, n=5, player="Carlsen,M"):
    ...
    return res
```

Expected result on the full dataset:

```
[('Firouzja,Alireza', 90), ('Nakamura,Hi', 61), ('Lazavik,Denis', 56),
 ('Niemann,Hans Moke', 46), ('Caruana,F', 43)]
```

4) Define a function named `longest_game` that finds the game with the most move numbers (the first one in file order if there is a tie), prints it in the format below and returns its dictionary. **[2 points]**

```
def longest_game(games):
    ...
    return game
```

The function must also print:

```
I have found the longest game:
Klukin,Kirill vs Carlsen,M, Titled Tue 13th Jan 2026 (2026.01.13): 148 moves
```

2 Exercise 2 [Part A]

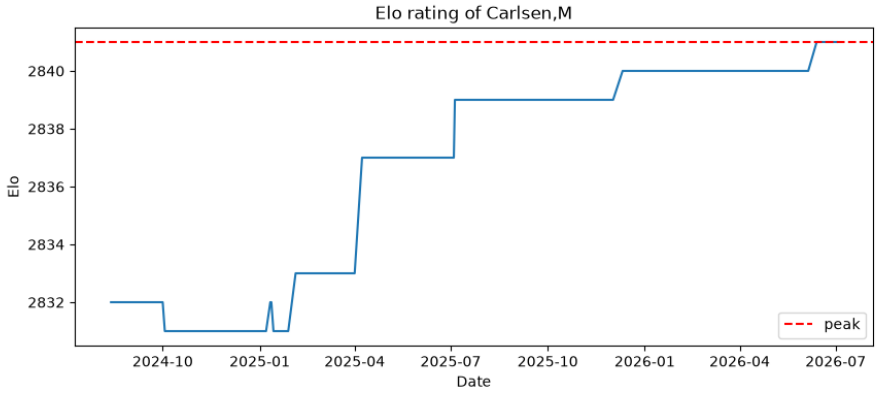
Write your code in `exercise2.py`, using `matplotlib`. The file already imports your `parse_games` function from `exercise1.py` to load the dataset.

Each function receives the list of games returned by `parse_games`, must create a *new* figure and must save it in the `figures` directory. Try to make your plots as close as possible to the examples

down below.

1) Define a function named `plot_elo_progress` that draws a line chart of `player`'s Elo rating over time. Use one point per date: the player's Elo in the first game of that date, in file order. Add a dashed horizontal line at the highest rating. **[3 points]**

```
def plot_elo_progress(games, player="Carlsen,M"):
    ...
```

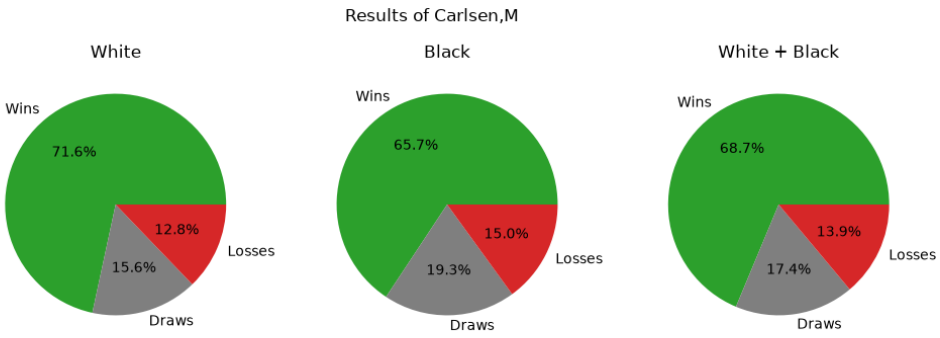


Hint: Convert each date string into a datetime object so that Matplotlib can interpret and display the dates correctly on the x-axis with this:

```
from datetime import datetime
converted_date = datetime.strptime(game["date"], "%Y.%m.%d").date()
```

2) Define a function named `plot_results_pies` that draws a figure with three pie charts side by side, showing `player`'s wins, draws and losses (in this order) for the games played as White, as Black, and in total. Title the pies `White`, `Black` and `White + Black`, label the slices `Wins`, `Draws` and `Losses`, and show the percentage on each slice. **[5 points]**

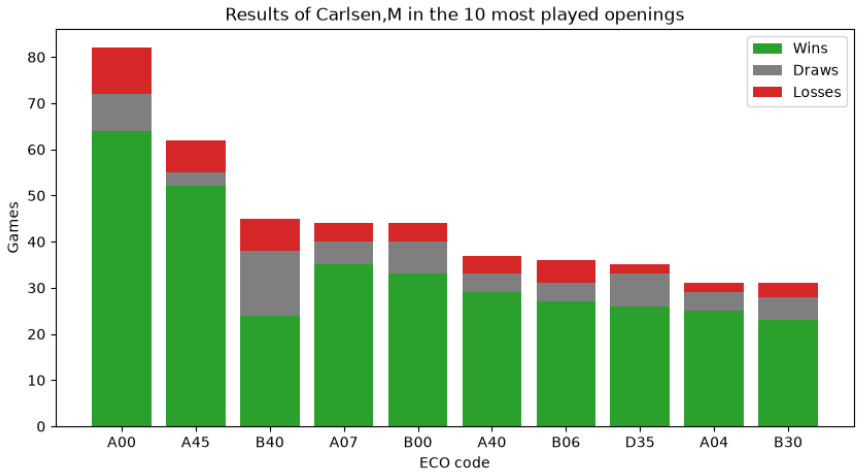
```
def plot_results_pies(games, player="Carlsen,M"):
    ...
```



3) Define a function named `plot_top_openings` that draws a stacked bar chart of the `n` most played openings (ECO codes). Sort the bars by number of games (highest first), breaking ties by ECO code in alphabetical order. Each bar stacks `player`'s wins at the bottom, then draws, then losses. Use the ECO codes as x tick labels and add a legend. **[6 points]**

```
def plot_top_openings(games, n=10, player="Carlsen,M"):
```

```
...
```


Results of Carlsen,M in the 10 most played openings
Wins
Draws
Losses
Games
0
10
20
30
40
50
60
70
80
A00
A45
B40
A07
B00
A40
B06
D35
A04
B30
ECO code